

Performance Improvement of Redundant Requests in Distributed Systems

Santa Maria Shithil
Stony Brook University
Stony Brook, NY, USA
sshithil@cs.stonybrook.edu

Muhammad Abdullah Adnan
Bangladesh University of Engineering and Technology
(BUET)
Dhaka, Bangladesh
abdullah.adnan@gmail.com

Abstract

The techniques of request duplication and request reissue are used to reduce tail latency in distributed systems by sending redundant read requests to multiple replicas. However, these strategies can fail if the duplicated or reissued requests target replicas already experiencing high latency, worsening the problem. To improve the effectiveness of these techniques, we propose implementing those on top of an efficient replica selection strategy. Experimental results demonstrate that our approach significantly enhances the performance of both techniques, enabling them to more effectively reduce tail latency in distributed systems. To evaluate the effectiveness of our strategy, we deployed a 15-node Cassandra cluster on Amazon EC2, spanning 5 geographical regions. We generated test datasets and workloads using the industry-standard Yahoo Cloud Serving Benchmark (YCSB).

CCS Concepts

• Computing methodologies;

Keywords

replica selection, request duplication, request reissue, redundant request, tail latency

ACM Reference Format:

Santa Maria Shithil and Muhammad Abdullah Adnan. 2024. Performance Improvement of Redundant Requests in Distributed Systems. In *11th International Conference on Networking, Systems, and Security (NSysS '24)*, December 19–21, 2024, Khulna, Bangladesh. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3704522.3704543>

1 Introduction

Geo-distributed systems reduce latency by storing data replicas in locations closer to the client. However, selecting the nearest replica does not always ensure optimal performance. Fluctuations in round trip time (RTT), server load, and network traffic can lead to long-tail latency, where a delay from just one slow replica can negatively impact overall system performance.

To address the tail latency problem, mainly three classes of strategies are used: 1) request duplication strategies 2) request reissuing

strategies and 3) replica selection strategies. In request duplication strategies [6, 8, 16, 18, 22] multiple copies of requests are dispatched to multiple replicas for every client's read request. The replicas process the request individually. Upon receiving the response from the fastest replica, the client discards the rest of the redundant requests. This technique is usually applied to systems with low system utilization. For systems that run at higher system utilization, one alternative approach is to use request reissuing techniques [2, 5, 6, 10, 21]. In this technique, initially, the client sends a single request to a single replica. If the request fails or encounters an error or does not respond for a specific time then the client will resend the request to the same replica or to a different replica. Another solution is to use replica selection strategies [9, 11, 12, 15, 17] where instead of sending multiple requests a single request is dispatched to the fastest replica. The selection of the fastest replica is done based on a score that is calculated depending on several factors including service time, queue size, latency, disk I/O operations, etc.

Although request reissuing and duplication techniques help reduce tail latency, the situation worsens when several reissued or duplicated requests target a replica that is already experiencing long-tail latency [19]. Therefore, special care should be taken to select replicas to route redundant requests of these techniques. Since it is evident that replica selection techniques can select the best replicas to route requests to, we can first use an efficient replica selection technique to select better replicas and then route redundant requests to those replicas.

In this research work, we aim to improve the performance of request duplication and request reissue strategies by implementing these alongside an efficient replica selection strategy. With this in mind, first, we developed an efficient replica selection strategy that predicts how quickly replicas will be able to respond to client requests. Based on this prediction, it will order and select the top N (number of replicas to which the redundant request will be routed) replicas. Then the request duplication and request reissue strategies will send the redundant requests to these N replicas. Since our proposed strategy will select faster replicas, it guarantees the performance improvement of the request duplication and request reissue strategies.

The rest of the paper is organized as follows. Section 2 discussed the state-of-the-art strategies. The motivation behind this research work is discussed in section 3. The detailed description of the system model is discussed in section 4. Section 5 described the implementation and the experimental setup. We discussed the detailed system evaluation in section 6. Finally, we concluded the paper in section 7.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

NSysS '24, December 19–21, 2024, Khulna, Bangladesh

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1158-9/24/12

<https://doi.org/10.1145/3704522.3704543>

2 Related Works

In this section, we discussed several state-of-the-art request duplication, request reissue, and replica selection strategies for reducing tail latency in distributed systems.

2.1 Request Duplication Strategies

In [8], Flach et al. present Proactive- recover from losses proactively which is a novel loss recovery mechanism for reducing web latency. This mechanism proactively transmits multiple copies of a single request and accepts the fastest response. In [16], Stewart et al. reduce the latency of storage access by using duplicate read/write request. To achieve salability, their proposed system copies the same data across multiple nodes. Read/write requests are sent to every duplicate, and receive only the fastest response. Vulimiri et al. in [18] also uses the concept of duplicate requests. They demonstrate that by introducing redundant requests to multiple resources and using the first result that is complete, the long tail latency can be reduced. Repflow [22] is a data center transport scheme for reducing latency for short TCP flows both on average and the 99th percentile. It replicates each short TCP flow by creating another TCP connection to the receiver. The applications use the fastest flow that finishes the transfer. Cassandra supports the request duplication strategy by sending two full read requests to two replicas and sending additional digest requests to other replicas depending on the consistency level [6].

2.2 Request Reissue Strategies

Kaler et. Al. in [10] introduces a new family of strategies that reissue a request after a delay d and a probability q . While other reissue strategies reissue a request after a certain delay, this strategy employs randomness to limit the rate of reissue but allows the request to be reissued as soon as possible so that it has enough time to respond. CostLO [21] reduces the latency variability associated with cloud storage services by reissuing requests so that the fastest response can be considered. It combines the forms of multiple redundancies by applying different configurations that are viable in practice. The Google Big Table [2] also uses reissue techniques for reducing tail latency. If it routes a read request to one cluster and that cluster is excessively slow to reply then it tries to reissue the request to another cluster. Cassandra also supports a request reissue strategy by sending one full read request and one or more digest requests to the replicas depending on the consistency level. If the replica to which a full request has been sent does not respond within a specific time interval then the coordinator sends another read request to another replica [6].

2.3 Replica Selection Strategies

Cassandra uses dynamic snitch [12] as a replica selection strategy. It calculates a score for each replica based on the latency information. Dynamic Snitch uses this score to select the best replica and routes the read request to that replica. In [17], L. Suresh et al. present C3 which is also a scoring based replica selection strategy. The score is calculated based on the queue size and service time of each replica. Heron [9] addressed the problem of tail latency for heterogeneous workloads. First, it classifies the read request as small and large using a bloom filter [1]. Then calculates a score for each of the

nodes of the cluster using the scoring mechanism of C3. Finally, it selects the best replica for routing the read request based on the score and the class information of the read request. In [11], W. Jiang et al. proposed a replica selection strategy that works in two steps. First chooses the replica servers corresponding to the least number of outstanding keys, and then sorts this subset of replica servers according to their response times. Jun Wu in [20], proposed a replica selection strategy that adopted the score calculation function of C3. Along with queue size and service time they also used waiting time in the queue to calculate the score.

3 Motivation

Request duplication and request reissue techniques are widely used to reduce tail latency in distributed systems. These techniques send redundant requests to multiple replicas. Generally, request duplication and request reissue strategies randomly select these replicas. However, if selected replicas are already suffering from tail latency issues then the situation becomes worse. Therefore, special care should be taken to select the replicas to route the read request. One solution is to use an efficient replica selection strategy to pick the fastest replicas to route redundant requests. Although several replica selection techniques have already been developed none of them are efficient enough to accommodate the dynamic nature of distributed systems and select the best replica. Replica selection techniques need to consider various factors such as RTT, response time, disk IO rate to select better replicas. However, none of the existing replica selection strategies discussed in the previous section considered all these important factors to decide on the best replica. Therefore, first we designed a new replica selection strategy that would be able to adapt to the dynamic nature by considering all important factors. Then we implemented the request duplication and request reissue strategy on top of our proposed replica selection strategy to improve the performance of these strategies.

4 System Model

We first design a prediction based replica selection strategy. We then use this strategy to pick top N replicas to send the redundant requests.

4.1 Prediction Based Replica Selection Strategy

To make the replica selection more efficient and adaptive to the dynamic nature of the distributed system we employed a prediction-based strategy. We designed a very simple prediction model with multiple linear regression. Our strategy first predict the latency of each of the servers using the prediction model. It then choose the server which has the lowest latency to send the read request.

4.1.1 Prediction Model. To design a prediction-based model, first we need to identify the independent variables for the linear regression. We should choose those factors as the independent variables that have the direct and significant impact on the latency of the geo-distributed systems. Such three most important factors are listed below.

- Response time: The response time of a server has linear relationship with the overall latency of a request. Since usually the response time is very low compared to the latency one

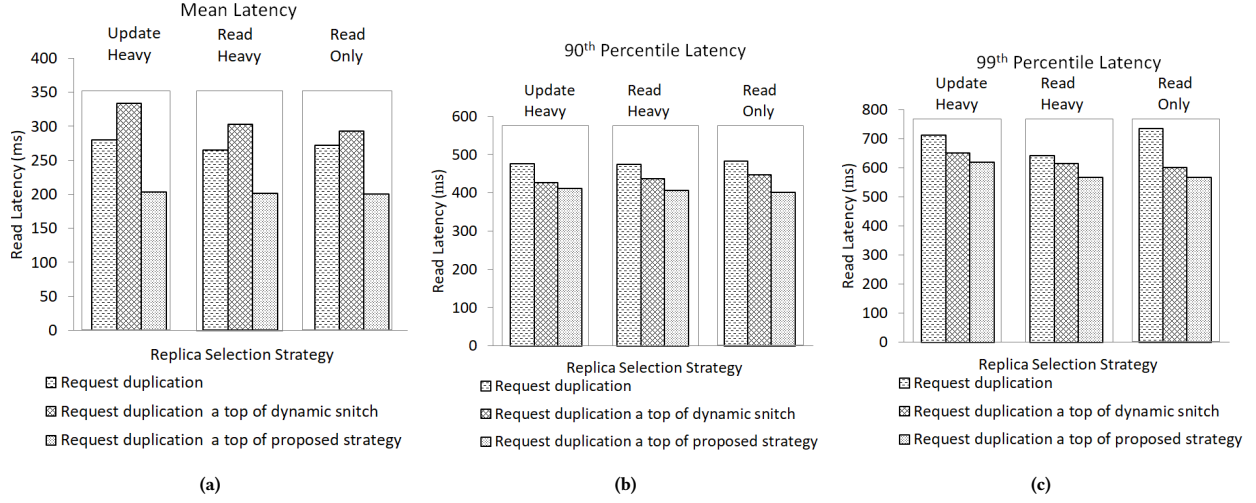


Figure 1: Latency of Cassandra when using request duplication, request duplication on top of dynamic snitch, and request duplication a top of our proposed strategy (a) mean latency (b) 90th percentile latency (c) 99th percentile latency.

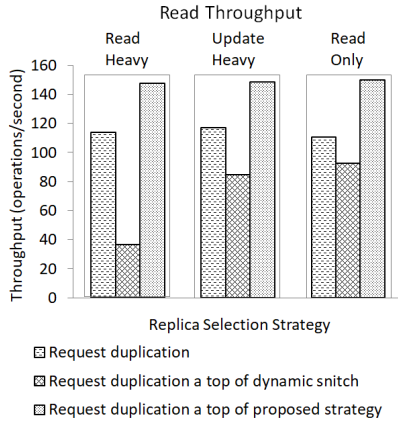


Figure 2: Comparison of read throughput at different workloads. Implementation of request duplication strategy on top of our proposed strategy shows higher throughput at all the scenarios.

may think that the response time can be ignored. However, with a heavy load situation the waiting time in the queue of the server may increase which may increase the response time significantly.

- **Round Trip Time (RTT):** The RTT of a replica read request is the amount of time it takes for a request to travel from a client to the server and send back the response to the client from the server [3]. In geo-distributed systems, since the nodes of a cluster are geographically dispersed it is very crucial to keep into consideration the RTT during designing a replica selection strategy.
- **Disk I/O Operations Rate (IOPS):** The disk read latency has a linear relationship with the overall latency of the read request in distributed systems [14]. The higher the disk IOPS

rate in the server, the higher the disk read latency. In a Cassandra [13] cluster, each node performs compaction periodically to merge multiple SSTables. This significantly increases disk input-output operations which in turn increases the overall latency of the node. Hence for designing an efficient replica selection strategy we also need to consider the impact of disk I/O operations of a server as a selection criterion.

Finally, we develop an equation of multiple linear regression that predicts the latency of each server by using the response time, RTT, and disk IOPS as the independent variables as shown in eq(1).

$$Y = a + b_1X_1 + b_2X_2 + b_3X_3 \quad (1)$$

Where, Y = predicted latency, X_1 = response time, X_2 = RTT, X_3 = disk IOPS, b_i = coefficients, a = Y - intercept.

The value of the coefficients b_1 , b_2 , and b_3 can be calculated by using (2).

$$B = A^{-1} \cdot C \quad (2)$$

Where,

$$B = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} \quad (3)$$

$$C = \begin{bmatrix} \sum x_1y \\ \sum x_2y \\ \sum x_3y \end{bmatrix} \quad (4)$$

$$A = \begin{bmatrix} \sum x_1^2 & \sum x_1x_2 & \sum x_1x_3 \\ \sum x_1x_2 & \sum x_2^2 & \sum x_2x_3 \\ \sum x_1x_3 & \sum x_2x_3 & \sum x_3^2 \end{bmatrix} \quad (5)$$

$$\sum x_i = \sum X_i^2 - \frac{(\sum X_i)^2}{n} \quad (6)$$

$$\sum x_iy = \sum X_iY - \frac{(\sum X_i)(\sum Y)}{n} \quad (7)$$

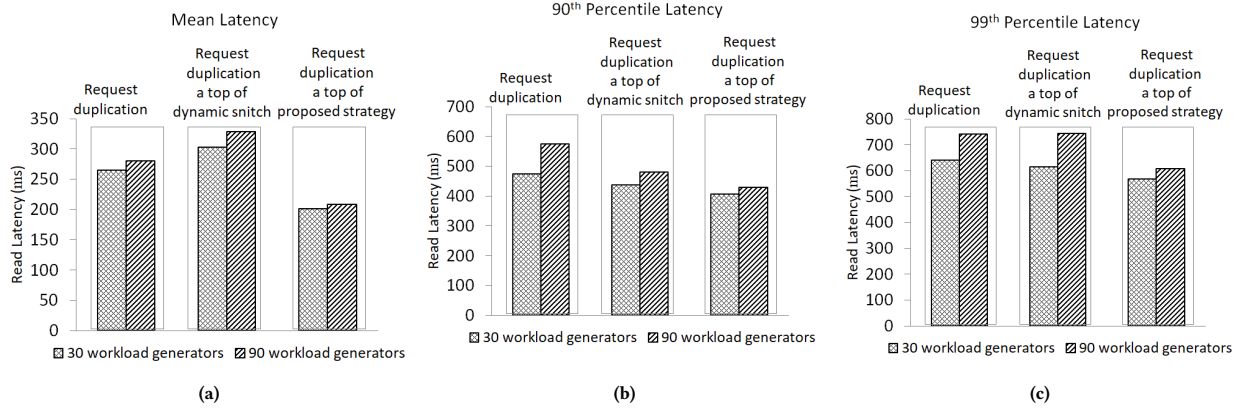


Figure 3: Latency of Cassandra when using request duplication, request duplication on top of dynamic snitch, and request duplication a top of our proposed strategy at a higher workload (a) mean latency (b) 90th percentile latency (c) 99th percentile latency.

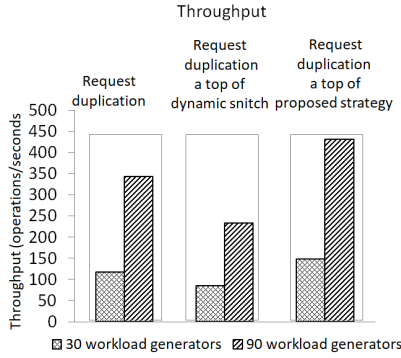


Figure 4: Throughput of Cassandra when using request duplication strategy, request duplication on top of dynamic snitch, and request duplication a top of our proposed strategy at higher workloads

$$\sum x_i x_j = \sum X_i X_j - \frac{(\sum X_i)(\sum X_j)}{n} \quad (8)$$

$$n = \text{number of entries} \quad (9)$$

The y-intercept a can be calculated using (10).

$$a = \bar{y} - b_1 \bar{x}_1 - b_2 \bar{x}_2 - b_3 \bar{x}_3 \quad (10)$$

Where

$$\bar{y} = \frac{\sum Y}{n} \quad \text{and} \quad \bar{x}_i = \frac{\sum X_i}{n} \quad (11)$$

During each read response the server sends response time and disk IOPS as feedback information to the clients whereas the RTT and latency are tracked by the clients themselves.

4.1.2 Smoothing the Predicted Latency. We derived (1) for predicting latency where we use the parameters response time, RTT, and disk IOPS directly. However, to make the prediction more accurate we use Exponentially Weighted Moving Average (EWMA) of all the

parameters instead of using these directly. We get a new smooth latency using (12).

$$\hat{Y} = a + b_1 \hat{X}_1 + b_2 \hat{X}_2 + b_3 \hat{X}_3 \quad (12)$$

4.2 Integrating Request Duplication and Request Reissue Strategy with the Replica Selection Strategy

To improve the performance of the request duplication and request reissue strategies we implemented it on top of our proposed prediction-based replica selection strategy as discussed in section 4.1. The replica selector first selects a subset of the replicas that are expected to respond to the read request within the shortest possible time. The selection is done based on the predicted latency. The system then uses this selected subset of replicas to send the redundant requests. The request duplication strategy sends two full read requests to two replicas and additional digest requests based on the consistency level. The request reissue strategy sends one full read request and one or more digest requests based on the consistency level. If the replica to which a full request has been sent does not respond within a specific Xpercentile time interval then the coordinator sends another read request to another replica.

5 Implementation and Experimental Setup

We have implemented our proposed prediction-based replica selection strategy in Cassandra 3.0. Our implementation consists of a total of 272 lines. We modified the `theupdateScores()` method of Cassandra and implemented our new score calculation technique. To track, calculate, and update the coefficients we have implemented a `coefficientUpdater()` method. We set the smoothing factor α to 0.8 to assign more weight to the recent data. We performed the recursive least square method to ensure the stability of our proposed regression model. Cassandra has the implementation of both request duplication and request reissue technique and we can use these techniques by configuring the `speculativeRetry` property. To enable the request duplication strategy we set the speculative retry

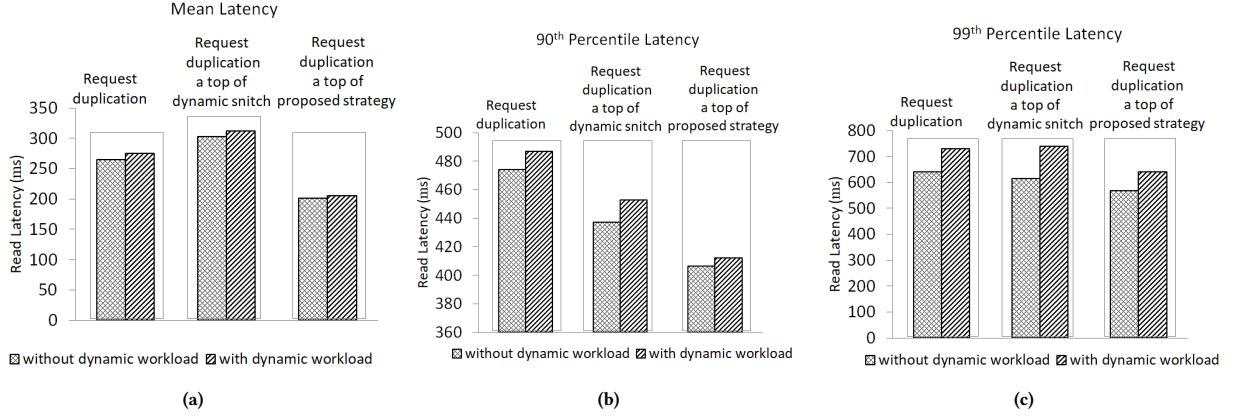


Figure 5: Latency of Cassandra when using request duplication, request duplication on top of dynamic snitch, and request duplication a top of our proposed strategy at a dynamic workload (a) mean latency (b) 90th percentile latency (c) 99th percentile latency.

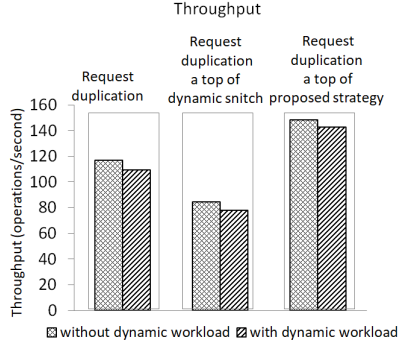


Figure 6: Throughput of Cassandra when using request duplication strategy, request duplication a top of dynamic snitch, and request duplication a top of our proposed strategy at dynamic workloads

property to *ALWAYS*. To enable the request reissue we configure the speculative retry to *99percentile*. It is the default setting of Cassandra and provides a good balance between not performing a lot of extra requests while still dealing with the worst problems [7].

To conduct our evaluation, we established a Cassandra cluster on Amazon EC2, employing 15 c5d.large instances. This cluster was distributed across five geographically dispersed racks, spanning locations in Singapore, US East (N. Virginia), Europe (Paris), Canada (Central), and Australia (Sydney). Each of these cluster nodes boasted 4GB of RAM and a 264GB hard disk capacity. We employed the Ubuntu 16.04 operating system for Cassandra cluster deployment. Our selected snitch was the *GossipingPropertyFileSnitch*, and we set the replication factor to 3. To ensure even token distribution among Cassandra nodes, we configured Numtoken as 256. Given the absence of real workloads, we utilized the YCSB [4] to generate datasets and execute workloads. Specifically, we inserted 100 million records, each with a 1KB length, all generated from the YCSB. To orchestrate the workloads directed toward the

Cassandra cluster, we utilized a separate c5d.large instance housing an instance of YCSB. This YCSB instance was equipped with 30 request generators, including 10 million read operations. Our workload distribution was based on the Zipfian distribution, utilizing the default Zipf parameter value. We designed three distinct types of workloads to comprehensively evaluate our system:

- **WorkloadA:** A balanced workload with equal reads and writes (50/50).
- **WorkloadB:** A read-intensive workload with a higher emphasis on reads (95/5 reads/writes).
- **WorkloadC:** A read-only workload, exclusively focusing on read operations (100/0 reads/writes).

This multifaceted approach allowed us to thoroughly assess the performance and resilience of our system under various operational scenarios.

6 System Evaluation

In this section, we evaluated the effect of our proposed prediction-based replica selection strategy on the request duplication and request reissue strategy.

6.1 Performance Evaluation of the Request Duplication Strategy

To evaluate the performance we implemented the request duplication strategy on top of both Cassandra’s dynamic snitch and our proposed strategy.

6.1.1 Comparison of Latency at Different Workload. We conduct this experiment to determine how replica selection strategy reduces the latency of the request duplication strategy. Fig. 1 shows that in almost all the scenarios the latency of the request duplication strategy has reduced when implemented on top of dynamic snitch and our proposed strategy. However, our proposed strategy reduced the latency more effectively compared to the dynamic snitch. The reason for such an improvement is that when we implement the request duplication strategy on top of a replica selection strategy, it

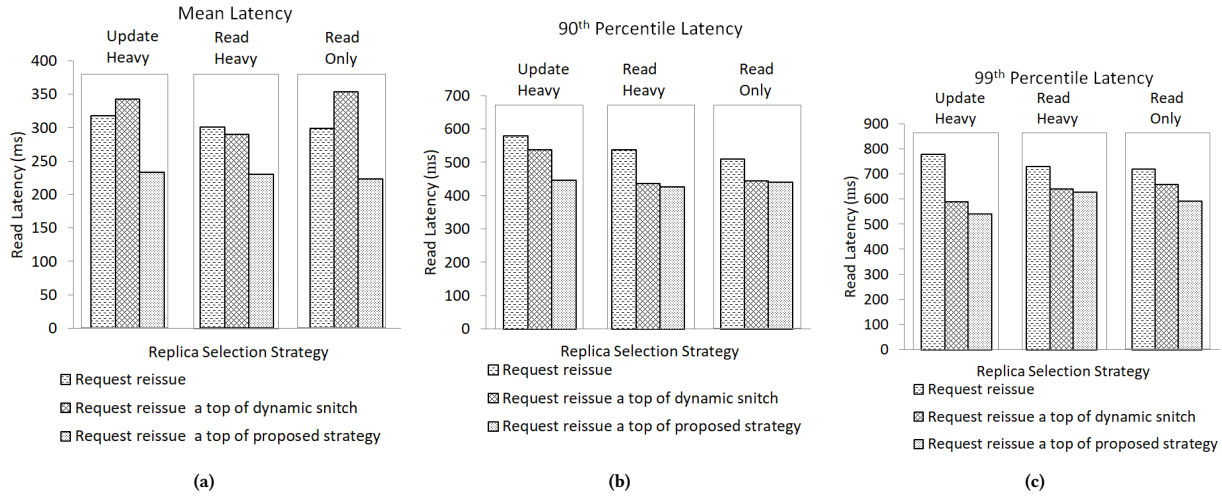


Figure 7: Latency of Cassandra when using request reissue, request reissue on top of dynamic snitch, and request reissue a top of our proposed strategy (a) mean latency (b) 90th percentile latency (c) 99th percentile latency.

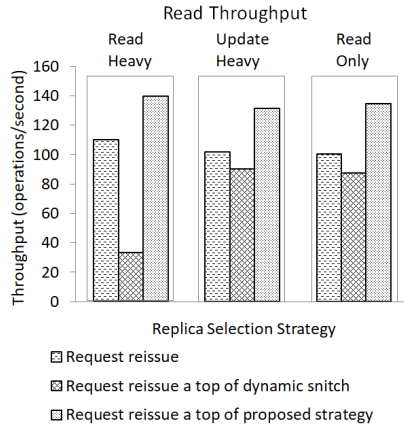


Figure 8: Comparison of read throughput at different workloads. Implementation of request reissue strategy on top of our proposed strategy shows higher throughput at all the scenarios.

sends the duplicate request to a sub-optimal subset of the replicas. As our proposed strategy selects better replicas compared to dynamic snitch the implementation of the request duplication strategy on top of our strategy shows the lowest latency.

6.1.2 Comparison of Throughput at Different Workload. We conducted this experiment to see whether the replica selection strategy adding any overhead to the request duplication strategy and thereby hampering the throughput. Fig.2 depicts the comparison of throughput of request duplication strategy with request duplication on top of dynamic snitch, and request duplication on top of our proposed strategy. In all the scenarios, the system has the highest throughput when we implemented request duplication on top of our replica selection strategy. By reducing latency across the replicas, the implementation of request duplication strategy on top

of our strategy makes better use of the system capacity and thereby increases the system throughput. However, the system observes the lowest throughput when implemented the request duplication strategy on top of the Cassandra dynamic snitch since it is less effective to replica selection compared to our strategy.

6.1.3 Performance at Higher Workload. We conduct this experiment to evaluate the performance of the request duplication strategy at a higher workload. For this experiment, we increase the number of workload generators from 30 to 90 and we conduct this experiment with only read-heavy workload. Fig. 3 shows, still at a higher workload situation the request duplication strategy has the lowest latency when implemented on top of our proposed replica selection strategy. Moreover, the request duplication strategy observes the lowest increase in latency when implemented on top of our strategy. Fig. 4 shows the throughput comparison at higher workload situation. We can see also at higher workload situations the throughput of the request duplication technique is the highest when implemented on top of our proposed strategy.

6.1.4 Performance at Dynamic Workload. We conduct this experiment to evaluate the performance of the request duplication strategy at dynamic workload situations. For this experiment, we initially start 30 workload generators with read-heavy workload and after 330 sec we start another 30 workload generators with update-heavy workload. Fig. 5 depicts that the performance of the request duplication strategy, request duplication on top of the dynamic snitch, and request duplication on top of our proposed strategy is quite identical to the performance in higher workload situations. We also compare the throughput in dynamic workload situation as shown in fig. 6. Unlike the higher workload situation where the throughput of all the strategies increases as the workload increases, in the dynamic workload situation the throughput of all the strategies decreases. However, the request duplication strategy observe the lowest decrease in throughput while implemented on top of our replica selection strategy.

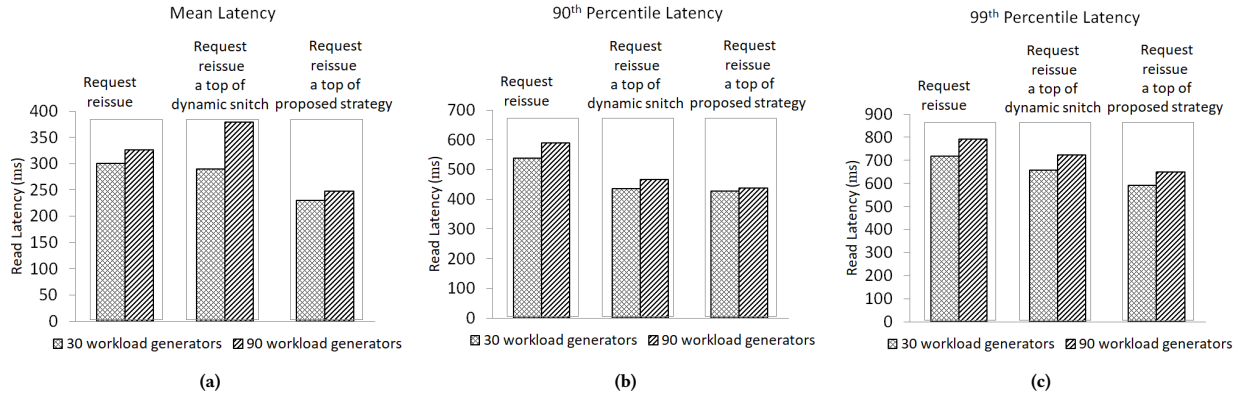


Figure 9: Latency of Cassandra when using request reissue, request reissue on top of dynamic snitch, and request reissue a top of our proposed strategy at a higher workload (a) mean latency (b) 90th percentile latency (c) 99th percentile latency.

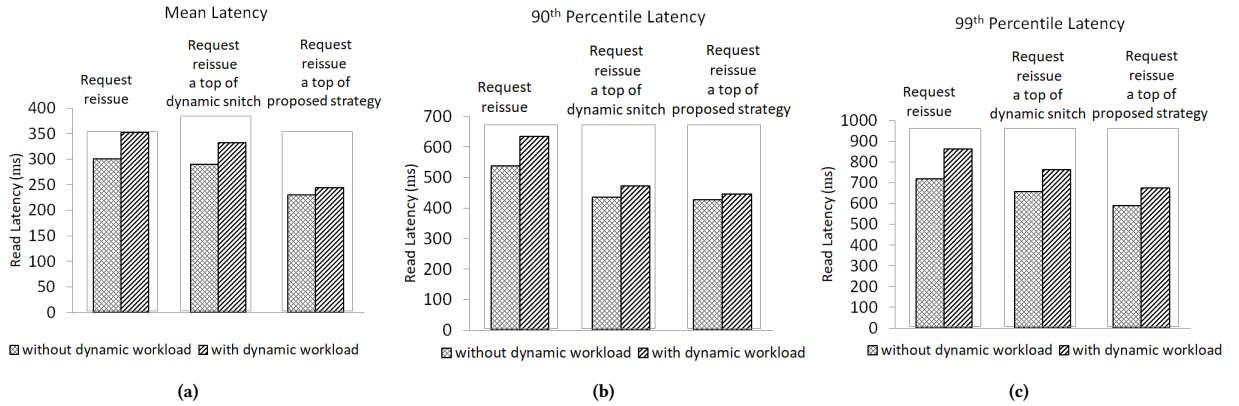


Figure 10: Latency of Cassandra when using request reissue, request reissue on top of dynamic snitch, and request reissue a top of our proposed strategy at a dynamic workload (a) mean latency (b) 90th percentile latency (c) 99th percentile latency.

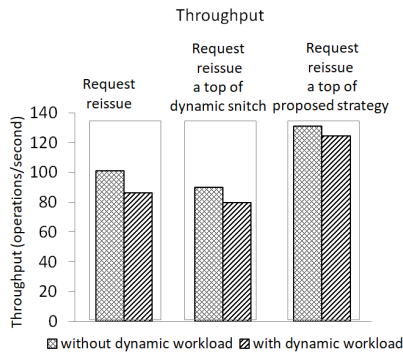


Figure 11: Throughput of Cassandra when using request reissue strategy, request reissue on top of dynamic snitch, and request reissue a top of our proposed strategy at dynamic workloads

6.2 Performance Evaluation of the Request Reissue Strategy

To evaluate the performance we also implemented the request duplication strategy on top of both Cassandra's dynamic snitch and our proposed strategy.

6.2.1 Comparison of Latency at Different Workload. We conduct this experiment to evaluate the performance of the request reissue strategy in different workload situation. Fig. 7 shows, like the request duplication, request reissue strategy also observes lowest latency in almost all the scenarios when implemented on top of our replica selection strategy.

6.2.2 Comparison of Throughput at Different Workload. Fig. 8 shows the comparison of throughput of the system when it uses the request reissue strategy, request reissue on top of the dynamic snitch, and request reissue on top of our proposed strategy. In all the scenarios the implementation of request reissue on top of our proposed replica selection strategy has the highest throughput. The reason for such an improvement is the reduction in latency as discussed in

section 6.2.1. Though our proposed strategy increases the throughput of the system, the dynamic snitch decreases it. We also observed the same outcome in the case of the request duplication strategy.

6.2.3 Performance at Higher Workload. We conduct this experiment to evaluate the performance of the request reissue strategy at a higher workload situation. To conduct this experiment, we increase the number of workload generators from 30 to 90 and use the read-heavy workload. Fig. 9 depicts, in all the scenarios, the request reissue strategy has the lowest increase in latency when implemented on top of our proposed replica selection strategy. Also, in a higher workload situation the request reissue strategy observes the highest throughput when implemented on top of our proposed strategy.

6.2.4 Performance at Dynamic Workload. Fig. 10 shows the comparison of latency at dynamic workload situation of request reissue, request reissue on top of our proposed strategy, and request reissue on top of the dynamic snitch. To conduct this experiment, we initially start 30 workload generators with a read-heavy workload and after 330 sec we start another 30 workload generators with an update-heavy workload. From fig. 10 we can see that as the system enters into the dynamic workload situation the latency increases. However, the increase in latency is the lowest when request reissue is implemented on top of our proposed strategy. Fig. 11 shows the throughput comparison of the request reissue strategy. The result is quite identical to the request duplication strategy. As the system enters into the dynamic workload situation the throughput of all the strategies decreases. However, the amount of decrease is the lowest when the request reissue strategy is implemented on top of our proposed strategy. Though the throughput decreases still at a dynamic workload situation the request reissue strategy has the highest throughput when implemented on top of our proposed strategy.

7 Conclusion

In this paper, a prediction-based replica selection strategy is proposed to improve the performance of request reissue and request duplication techniques by intelligently routing reissued or duplicated requests. The model predicts server latency and ranks servers to optimize request routing. Experimental results show that this approach enhances the performance of both strategies. However, Cassandra's dynamic snitch did not consistently improve performance and sometimes led to worse results. We can conclude that only an efficient replica selection strategy can reliably improve the effectiveness of these techniques, while an inefficient one may degrade performance.

References

- [1] Burton H. Bloom. 1970. Space/time trade-offs in hash coding with allowable errors. *Commun. ACM* 13, 7 (jul 1970), 422–426. <https://doi.org/10.1145/362686.362692>
- [2] Fay Chang, Jeffrey Dean, Sanjay Ghemawat, Wilson C. Hsieh, Deborah A. Wallach, Mike Burrows, Tushar Chandra, Andrew Fikes, and Robert E. Gruber. 2008. Bigtable: A Distributed Storage System for Structured Data. *ACM Trans. Comput. Syst.* 26, 2, Article 4 (jun 2008), 26 pages. <https://doi.org/10.1145/1365815.1365816>
- [3] Cloudflare. 2021. What is Round-Trip Time? | RTT Definition. <https://www.cloudflare.com/en-gb/learning/cdn/glossary/round-trip-time-rtt/>. Accessed: 10-Sep-2024.
- [4] Brian F. Cooper, Adam Silberstein, Erwin Tam, Raghu Ramakrishnan, and Russell Sears. 2010. Benchmarking cloud serving systems with YCSB. In *Proceedings of the 1st ACM Symposium on Cloud Computing* (Indianapolis, Indiana, USA) (SoCC '10). Association for Computing Machinery, New York, NY, USA, 143–154. <https://doi.org/10.1145/1807128.1807152>
- [5] DataStax. 2024. Cassandra Client Requests Read. <https://docs.datastax.com/en/cassandra-oss/3.x/cassandra/dml/dmlClientRequestsRead.html>. Accessed: 2024-09-17.
- [6] DataStax. 2024. CQL Create Table Reference. https://docs.datastax.com/en/cql-oss/3.3/cql/cql_reference/cqlCreateTable.html. Accessed: 2024-09-17.
- [7] DataStax Documentation. 2022. How are Read Requests Accomplished? <https://docs.datastax.com/en/cassandra-oss/3.x/cassandra/dml/dmlClientRequestsRead.html>. Accessed: 10-Sep-2024.
- [8] Tobias Flach, Nandita Dukkupati, Andreas Terzis, Barath Raghavan, Neal Cardwell, Yuchung Cheng, Ankur Jain, Shuai Hao, Ethan Katz-Bassett, and Ramesh Govindan. 2013. Reducing web latency: the virtue of gentle aggression. *SIGCOMM Comput. Commun. Rev.* 43, 4 (aug 2013), 159–170. <https://doi.org/10.1145/2534169.2486014>
- [9] Vikas Jaiman, Sonia Ben Mokhtar, Vivien Quéma, Lydia Y. Chen, and Etienne Rivière. 2018. Héron: Taming Tail Latencies in Key-Value Stores Under Heterogeneous Workloads. In *2018 IEEE 37th Symposium on Reliable Distributed Systems (SRDS)*. 191–200. <https://doi.org/10.1109/SRDS.2018.00030>
- [10] Virajith Jalaparti, Peter Bodik, Srikanth Kandula, Ishai Menache, Mikhail Rybalkin, and Chenyu Yan. 2013. Speeding up distributed request-response workflows. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM* (Hong Kong, China) (SIGCOMM '13). Association for Computing Machinery, New York, NY, USA, 219–230. <https://doi.org/10.1145/2486001.2486028>
- [11] Wanchun Jiang, Haiming Xie, Xiangqian Zhou, Liyuan Fang, and Jianxin Wang. 2017. Performance Analysis and Improvement of Replica Selection Algorithms for Key-Value Stores. In *2017 IEEE 10th International Conference on Cloud Computing (CLOUD)*. 786–789. <https://doi.org/10.1109/CLOUD.2017.115>
- [12] Avinash Lakshman and Prashant Malik. 2010. Cassandra: A Decentralized Structured Storage System. *ACM SIGOPS Operating Systems Review* 44, 2 (2010), 35–40. <https://doi.org/10.1145/1773912.1773922>
- [13] Avinash Lakshman and Prashant Malik. 2010. Cassandra: A Decentralized Structured Storage System. *ACM SIGOPS Operating Systems Review* 44, 2 (2010), 35–40. <https://doi.org/10.1145/1773912.1773922>
- [14] Steven Larsen and Ben Lee. 2014. Survey on System I/O Hardware Transactions and Impact on Latency, Throughput, and Other Factors. *Advances in Computers* 92 (2014), 67–104. <https://doi.org/10.1016/B978-0-12-420232-0.00002-7>
- [15] Santa Maria Shithil and Muhammad Abdullah Adnan. 2020. A Prediction Based Replica Selection Strategy for Reducing Tail Latency in Distributed Systems. In *2020 IEEE 13th International Conference on Cloud Computing (CLOUD)*. 522–526. <https://doi.org/10.1109/CLOUD49709.2020.00078>
- [16] Christopher Stewart, Aniket Chakrabarti, and Rean Griffith. 2013. Zoolander: Efficiently Meeting Very Strict, Low-Latency SLOs. In *10th International Conference on Autonomic Computing (ICAC 13)*. USENIX Association, San Jose, CA, 265–277. <https://www.usenix.org/conference/icac13/technical-sessions/presentation/stewart>
- [17] Lalith Suresh, Marco Canini, Stefan Schmid, and Anja Feldmann. 2015. C3: Cutting Tail Latency in Cloud Data Stores via Adaptive Replica Selection. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. USENIX Association, Oakland, CA, 513–527. <https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/suresh>
- [18] Ashish Vulimiri, Oliver Michel, P. Brighten Godfrey, and Scott Shenker. 2012. More is less: reducing latency via redundancy. In *Proceedings of the 11th ACM Workshop on Hot Topics in Networks* (Redmond, Washington) (HotNets-XI). Association for Computing Machinery, New York, NY, USA, 13–18. <https://doi.org/10.1145/2390231.2390234>
- [19] Ashish Vulimiri, Oliver Michel, P. Brighten Godfrey, and Scott Shenker. 2012. More is less: reducing latency via redundancy. In *Proceedings of the 11th ACM Workshop on Hot Topics in Networks* (Redmond, Washington) (HotNets-XI). Association for Computing Machinery, New York, NY, USA, 13–18. <https://doi.org/10.1145/2390231.2390234>
- [20] Jun Wu. 2018. *Reducing the Tail Latency of a Distributed NoSQL Database*. Ph.D. Dissertation. University of Nebraska - Lincoln.
- [21] Zhe Wu, Curtis Yu, and Harsha V. Madhyastha. 2015. CosTLO: Cost-Effective Redundancy for Lower Latency Variance on Cloud Storage Services. In *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)*. USENIX Association, Oakland, CA, 543–557. <https://www.usenix.org/conference/nsdi15/technical-sessions/presentation/wu>
- [22] Hong Xu and Baochun Li. 2013. RepFlow: Minimizing Flow Completion Times with Replicated Flows in Data Centers. In *14th IEEE Conference on Computer Communications (INFOCOM)* (Toronto, ON, Canada). IEEE, 1581–1589.

Received 30 September 2024; accepted 4 November 2024